PROJECT : 16-Bit RISC Processor

NAME : E HARSHAVARDHAN GOUD
COLLEGE : RGUKT BASAR

DESIGN :

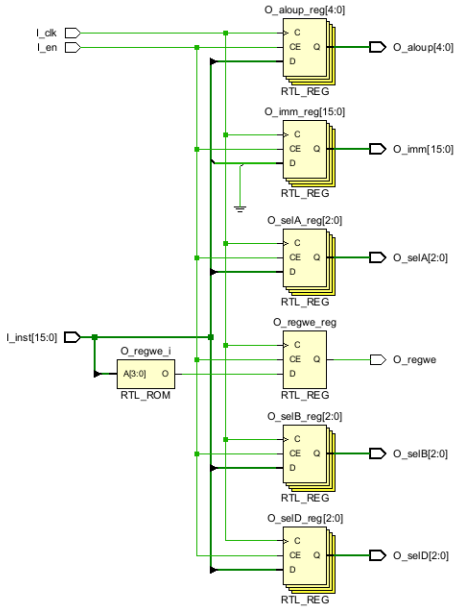
instruction_decoder :

instruction_decoder.v

C:/Users/user/OneDrive/Desktop/OneStop/RISC Processor/RISC Processor.srcs/sources_1/new/instruction_decoder.v

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company: RGUKT BASAR
// Engineer: E HARSHAVARDHAN GOUD
//
// Create Date: 28.08.2025 11:29:25
// Design Name: iNSTRUCTION DECODER
// Module Name: instruction_decoder
// Project Name: 16-BIT RISC PROCESSOR
// Description: Its main function is to translate the binary instruction
//              (opcode + operands) into control signals that guide the datapath components (ALU, registers, memory, etc.).
//////////////////////////////////////////////////////////////////////////////////


module instruction_decoder(
    input  I_clk,
    input  I_en,
    input  [15:0] I_inst,
    output reg [4:0] O_aloup,
    output reg [2:0] O_selA,
    output reg [2:0] O_selB,
    output reg [2:0] O_selD,
    output reg [15:0] O_imm,
    output reg O_regwe
);

    // Initialize outputs
    initial begin
        O_aloup <= 0;
        O_selA  <= 0;
        O_selB  <= 0;
        O_selD <= 0;
        O_imm   <= 0;
        O_regwe <= 0;
    end


    always @(posedge I_clk) begin
        if (I_en) begin
            // Example instruction decoding:
            // Assuming instruction format:
            // [15:11] opcode | [10:8] ra | [7:5] rb | [4:2] rd
            O_aloup <= I_inst[15:11] ;
            O_selA  <= I_inst[10:8] ;
            O_selB  <=I_inst [7:5] ;
            O_selD <= I_inst[4:2] ;
            O_imm   <= I_inst[7:0] ;
            case (I_inst[15:12])
                4'b0111:O_regwe <= 0;
                4'b1100:O_regwe <= 0;
                4'b1101:O_regwe <= 0;
                default:O_regwe <= 1;
            endcase
        end
    end
endmodule
```

SCHEMATIC :



Activate Windows
Go to Settings to activate Windows.

control_unit :

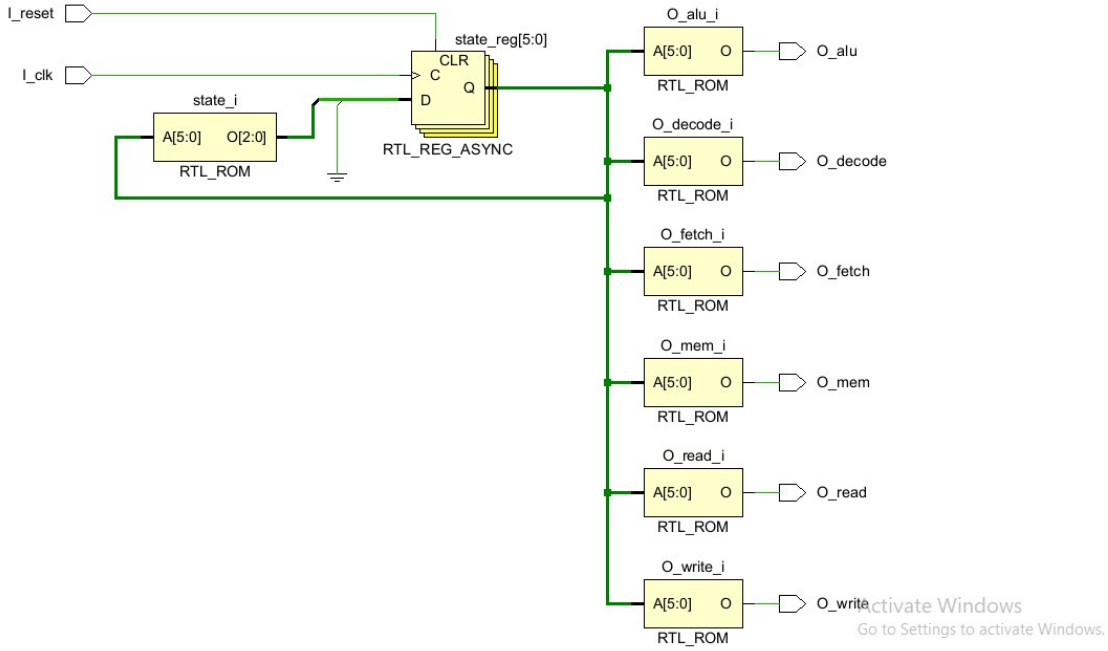
control_unit.v

C:/Users/user/OneDrive/Desktop/OneStop/RISC Processor/RISC Processor.srcs/sources_1/new/control_unit.v

```verilog
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company: RGUKT BASAR
// Engineer: E HARSHAVARDHAN GOUD
//
// Create Date: 28.08.2025 11:56:55
// Design Name: CONTROL UNIT
// Module Name: control_unit
// Project Name: 16 BIT RISC PROCESSOR

// Description: The Control Unit (CU) in a RISC (Reduced Instruction Set Computer) processor is responsible for:

//              1.Interpreting instructions fetched from memory.

//              2.Generating control signals that coordinate the datapath, ALU, registers, and memory.

//              3.Ensuring sequential and correct execution of instructions.

//////////////////////////////////////////////////////////////////////////////////

`timescale 1ns / 1ps

module control_unit(
    input I_clk,
    input I_reset,
    output reg O_fetch,
    output reg O_decode,
    output reg O_read,
    output reg O_alu,
    output reg O_write,
    output reg O_mem
);

    // Define states
    localparam s0 = 0,  // Fetch
               s1 = 1,  // Decode
               s2 = 2,  // Read
               s3 = 3,  // ALU
               s4 = 4,  // Write Back
               s5 = 5;  // Memory

    reg [5:0] state;

    // Initial state
    initial begin
        state <= s0;
    end

    // State transition
    always @(posedge I_clk or posedge I_reset) begin
        if (I_reset)
            state <= s0;   // reset back to fetch
        else begin
            case (state)
                s0: state <= s1; // Fetch → Decode
                s1: state <= s2; // Decode → Read
                s2: state <= s3; // Read → ALU
                s3: state <= s4; // ALU → Writeback
                s4: state <= s5; // Writeback → Memory
                default: state <= s0;
            endcase
        end
    end

    // Output logic
    always @(*) begin
        // Default off
        O_fetch  = 0;
```

```
        O_decode  = 0;
        O_read    = 0;
        O_alu     = 0;
        O_write    = 0;
        O_mem     = 0;

        case (state)
            s0: O_fetch   = 1;   // Fetch active
            s1: O_decode  = 1;   // Decode active
            s2: O_read    = 1;   // Execute active
            s3: O_alu     = 1;   // Mem active
            s4: O_write   = 1;   // Writeback active
            s5: O_mem     = 1;   // Memory
        endcase
    end

endmodule
```

SCHEMATIC :



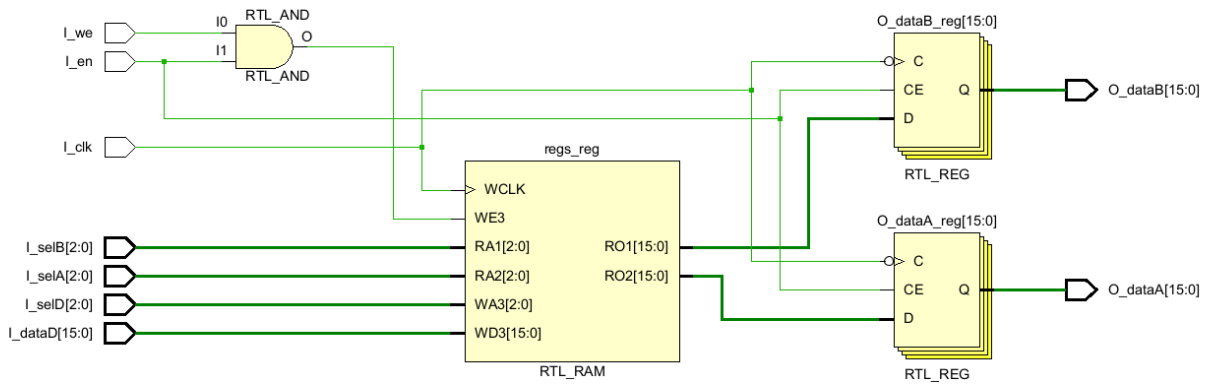
reg_file :

reg_file.v

C:/Users/user/OneDrive/Desktop/OneStop/RISC Processor/RISC Processor.srcs/sources_1/new/reg_file.v

```
`timescale 1ns / 1ps
module reg_file(
    input        I_clk,
    input        I_en,
    input        I_we,
    input  [2:0] I_selA,
    input  [2:0] I_selB,
    input  [2:0] I_selD,
    input  [15:0] I_dataD,
    output reg [15:0] O_dataA,
    output reg [15:0] O_dataB
);

    reg [15:0] regs [7:0];   // 8 registers, each 16-bit wide
    integer count;

    // Initialize register file
    initial begin
        O_dataA <= 0;
        O_dataB <= 0;
        for (count = 0; count < 8; count = count + 1) begin
            regs[count] <= 0;
        end
    end
    // Register file operation
    always @(negedge I_clk) begin
        if (I_en) begin
            if (I_we) begin
                regs[I_selD] <= I_dataD;  // Write data to destination register
            end
            O_dataA <= regs[I_selA];      // Read data from register A
            O_dataB <= regs[I_selB];      // Read data from register B
        end
    end
endmodule
```

SCHEMATIC :



Activate Windows
Go to Settings to activate Windows.

alu :

alu.v

C:/Users/user/OneDrive/Desktop/OneStop/RISC Processor/RISC Processor.srcs/sources_1/new/alu.v

```
`timescale 1ns / 1ps


module alu(
    input         I_clk,
    input         I_en,
    input  [4:0]  I_aluop,
    input  [15:0] I_dataA,
    input  [15:0] I_dataB,
    input  [7:0]  I_imm,
    output [15:0] O_dataResult,
    output reg    O_shldBranch
);

    reg [17:0] int_result;
    wire op_lsb;
    wire [3:0] opcode;

    // ALU operation codes
    localparam Add    = 0,
               Sub    = 1,
               OR     = 2,
               AND    = 3,
               XOR    = 4,
               NOT    = 5,
               Load   = 6,
               Cmp    = 9,
               SHL    = 10,
               SHR    = 11,
               JMPA   = 12,
               JMPR   = 13;

    initial begin
        int_result <= 0;
    end
```

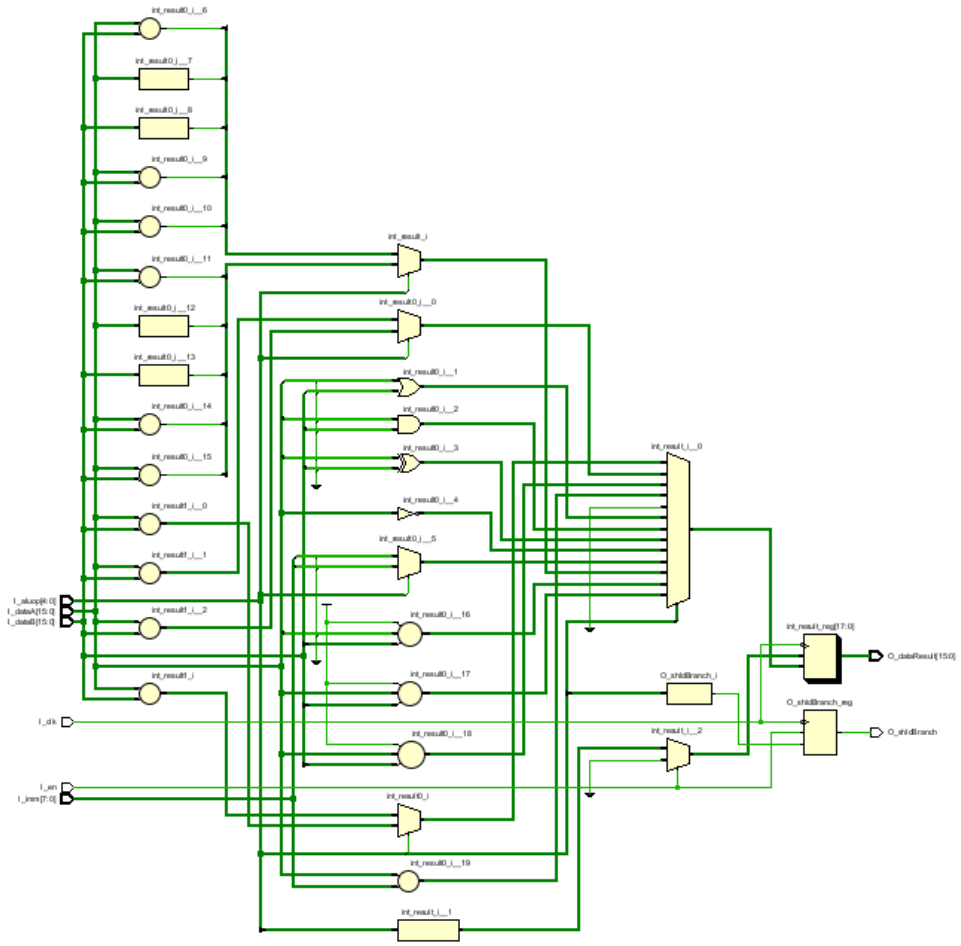
```

    assign op_lsb       = I_aluop[0];
    assign opcode       = I_aluop[4:1];
    assign O_dataResult = int_result[15:0];

    always @(negedge I_clk) begin
        if (I_en) begin
            case (opcode)

                Add: begin
                    int_result   <= (op_lsb ? ($signed(I_dataA) + $signed(I_dataB)): (I_dataA + I_dataB));
                    O_shldBranch <= 0;
                end

                Sub: begin
                    int_result   <= (op_lsb ? ($signed(I_dataA) - $signed(I_dataB)): (I_dataA - I_dataB));
                    O_shldBranch <= 0;
                end

                OR: begin
                    int_result   <= (I_dataA | I_dataB);
                    O_shldBranch <= 0;
                end

                AND: begin
                    int_result   <= (I_dataA & I_dataB);
                    O_shldBranch <= 0;
                end

                XOR: begin
                    int_result   <= (I_dataA ^ I_dataB);
                    O_shldBranch <= 0;
                end
```

```
                NOT: begin
                    int_result   <= ~I_dataA;
                    O_shldBranch <= 0;
                end

                Load: begin
                    int_result   <= (op_lsb ? {I_imm, 8'h00} : {8'h00, I_imm});
                    O_shldBranch <= 0;
                end

                Cmp: begin
                    if (op_lsb) begin
                        int_result[0] <= ($signed(I_dataA) == $signed(I_dataB)) ? 1 : 0;
                        int_result[1] <= ($signed(I_dataA) == 0) ? 1 : 0;
                        int_result[2] <= ($signed(I_dataB) == 0) ? 1 : 0;
                        int_result[3] <= ($signed(I_dataA) > $signed(I_dataB)) ? 1 : 0;
                        int_result[4] <= ($signed(I_dataA) < $signed(I_dataB)) ? 1 : 0;
                    end else begin
                        int_result[0] <= (I_dataA == I_dataB) ? 1 : 0;
                        int_result[1] <= (I_dataA == 0) ? 1 : 0;
                        int_result[2] <= (I_dataB == 0) ? 1 : 0;
                        int_result[3] <= (I_dataA > I_dataB) ? 1 : 0;
                        int_result[4] <= (I_dataA < I_dataB) ? 1 : 0;
                    end
                    O_shldBranch <= 0;
                end

                SHL: begin
                    int_result   <= I_dataA << I_dataB[3:0];
                    O_shldBranch <= 0;
                end

                SHR: begin
                    int_result   <= I_dataA >> I_dataB[3:0];
```


```
                    int_result   <= I_dataA >> I_dataB[3:0];
```


```
                    O_shldBranch <= 0;
                end

                JMPA: begin
                    int_result   <= $signed(I_dataA) >>> I_dataB[3:0];
                    O_shldBranch <= 0;
                end

                JMPR: begin
                    // Jump register -> result = A + immediate (branch decision)
                    int_result   <= I_dataA + {{8{I_imm[7]}}, I_imm};
                    O_shldBranch <= 1;   // enable branch signal
                end

                default: begin
                    int_result   <= 0;
                    O_shldBranch <= 0;
                end

            endcase
        end
    end

endmodule
```

SCHEMATIC :


int_result0_i__6
int_result0_i__7
int_result0_i__8
int_result0_i__9
int_result0_i__10
int_result0_i__11
int_result0_i__12
int_result0_i__13
int_result0_i__14
int_result0_i__15
int_result1_i__0
int_result1_i__1
I_aluop[4:0]
I_dataA[15:0]
I_dataB[15:0]
int_result1_i__2
int_result1_i
I_clk
I_en
I_ins[7:0]
int_result_i
int_result0_i__0
int_result0_i__1
int_result0_i__2
int_result0_i__3
int_result0_i__4
int_result0_i__5
int_result0_i__16
int_result0_i__17
int_result0_i__18
int_result0_i
int_result0_i__19
int_result_i__1
int_result_i__0
int_result_reg[17:0]
O_dataResult[15:0]
O_shldBranch_i
O_shldBranch_reg
int_result_i__2
O_shldBranch

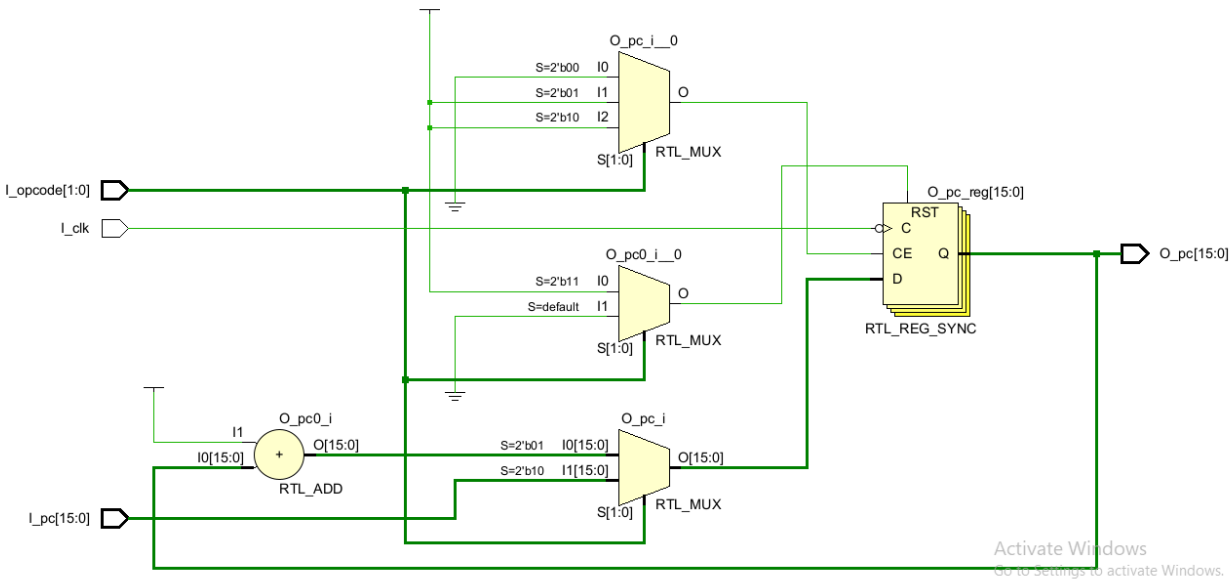
pc_unit:

pc_unit.v

C:/Users/user/OneDrive/Desktop/OneStop/RISC Processor/RISC Processor.srcs/sources_1/new/pc_unit.v

```
`timescale 1ns / 1ps

`timescale 1ns / 1ps

module pc_unit(
    input        I_clk,
    input  [1:0] I_opcode,
    input  [15:0] I_pc,
    output reg [15:0] O_pc
);

    initial begin
        O_pc <= 0;
    end

    always @(negedge I_clk) begin
        case (I_opcode)
            2'b00 : O_pc <= O_pc;        // Hold current PC
            2'b01 : O_pc <= O_pc + 1;    // Increment PC
            2'b10 : O_pc <= I_pc;        // Load PC from input
            2'b11 : O_pc <= 0;           // Reset PC
        endcase
    end

endmodule
```

SCHEMATIC :



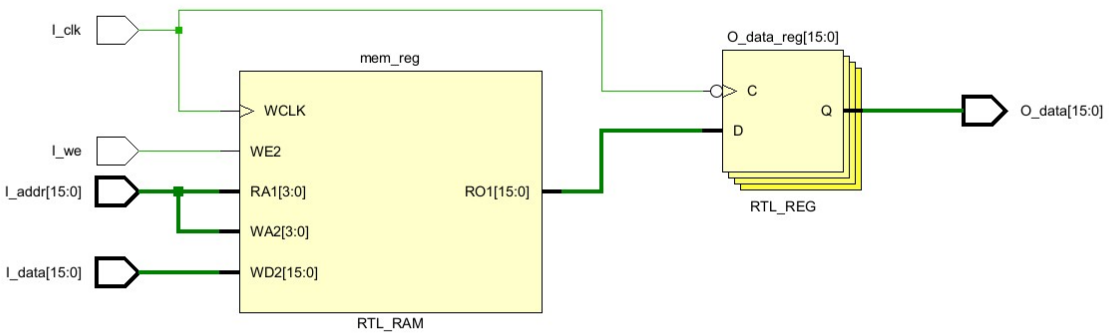
fake_ram :

fake_ram.v

C:/Users/user/OneDrive/Desktop/OneStop/RISC Processor/RISC Processor.srcs/sources_1/new/fake_ram.v

```
`timescale 1ns / 1ps

module Fake_ram(
    input        I_clk,
    input        I_we,
    input  [15:0] I_addr,
    input  [15:0] I_data,
    output reg [15:0] O_data
);

    reg [15:0] mem [8:0];   // 9 locations of 16-bit memory

    initial begin
        mem[0] = 16'b1000000001111110;
        mem[1] = 16'b100100000001101;
        mem[2] = 16'b0000000000000000;
        mem[3] = 16'b0000000000000001;
        mem[4] = 16'b0000000000000001;
        mem[5] = 16'b1000000000010000;
        mem[6] = 16'b100000000001001;
        mem[7] = 0;
        mem[8] = 0;
        O_data = 16'b0000000000000000;
    end

    always @(negedge I_clk) begin
        if (I_we) begin
            mem[I_addr[15:0]] <= I_data;   // Write data to memory
        end
        O_data <= mem[I_addr[15:0]];       // Read data from memory
    end

endmodule
```

SCHEMATIC :



Activate Windows
Go to Settings to activate Windows.

TEST BENCH :

decoder_unitest :

decoder_unitest.v

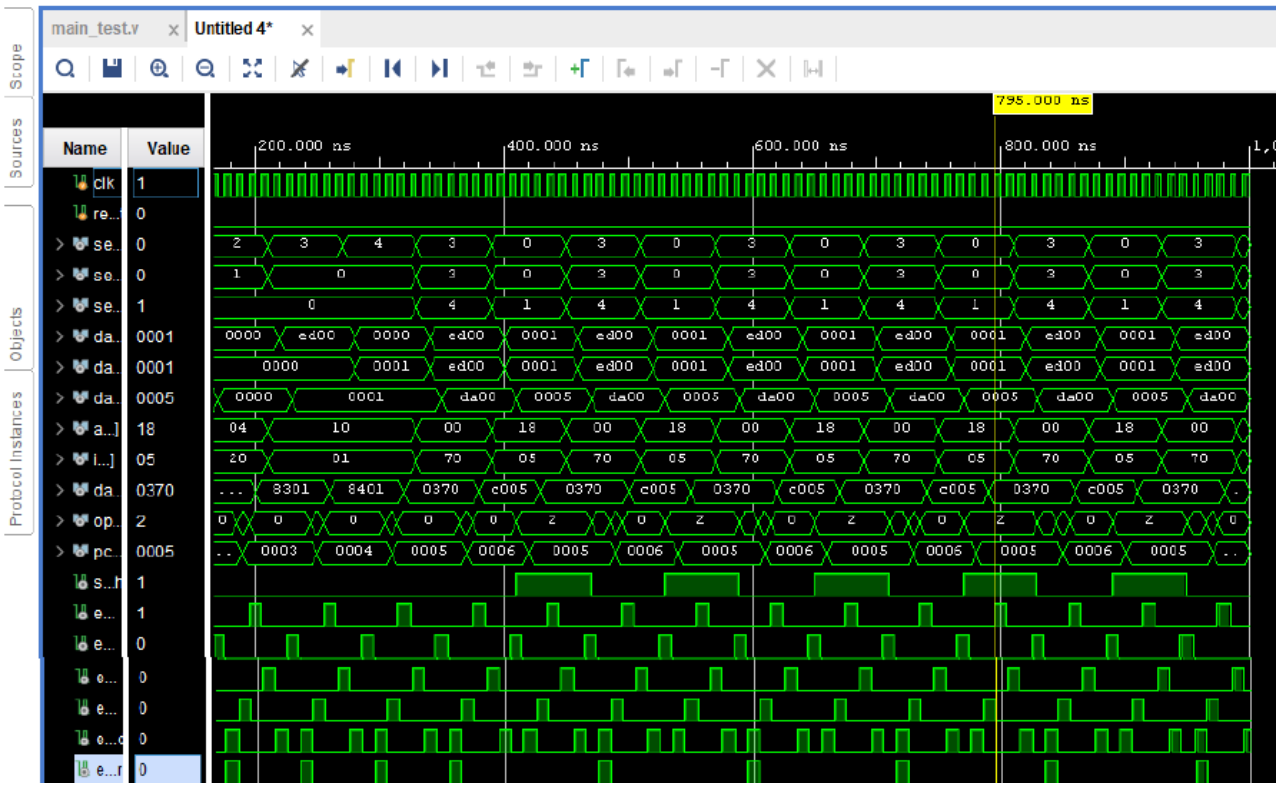
C:/Users/user/OneDrive/Desktop/OneStop/RISC Processor/RISC Processor.srcs/sim_1/new/decoder_unitest.v

```
`timescale 1ns / 1ps


module decoder_unitest();
    reg clk;
    reg reset;
    reg ram_we =0;
    reg [15:0] dataI=0;

    wire [2:0] selD;
    wire [2:0] selA;
    wire [2:0] selB;
    wire [15:0] dataA;
    wire [15:0] dataB;
    wire [15:0] dataD;
    wire [15:0] dataO;
    wire [7:0] imm;
    wire [4:0] aluop;
    wire [1:0]  opcode;
    wire [15:0] pco;

    wire shldBranch;
    wire enFetch;
    wire enalu;
    wire endec;
    wire enmem;
    wire enrgrd;
    wire enrgrw;
    wire regwe;
    wire update;
    assign enrgrw= regwe & update;
    assign opcode = (reset) ? 2'b11 : ((shldBranch) ? 2'b10 : (enmem) ? 2'b01 : 2'b00);
reg_file main_reg(
        clk,
        enrgrd,
```

```
        enrgwr,
        selA,
        selB,
        selD,
        dataD,
        dataA,
        dataB
    );
instruction_decoder main_instruction(
        clk,
        endec,
        dataO,
        aluop,
        selA,
        selB,
        selD,
        imm,
        regwe
    );
alu main_alu(
        clk,
        enalu,
        aluop,
        dataA,
        dataB,
        imm,

        dataD,
        shldBranch
    );

control_unit main_control(
        clk,
        reset,
        enfetch,
```

```
        endec,
        enrgrd,
        enalu,
        update,
        enmem
    );

    pc_unit pc_main(
        clk,
        opcode,
        dataD,
        pco
    );

    Fake_ram main_ram(
        clk,
        ram_we,
        pco,
        dataI,
        dataO
    );

    initial begin
        clk = 0;
        reset = 1;
        #20;
        reset = 0;
    end

    always begin
        #5;
        clk = ~clk;
    end

endmodule
```

WAVE FORMS :



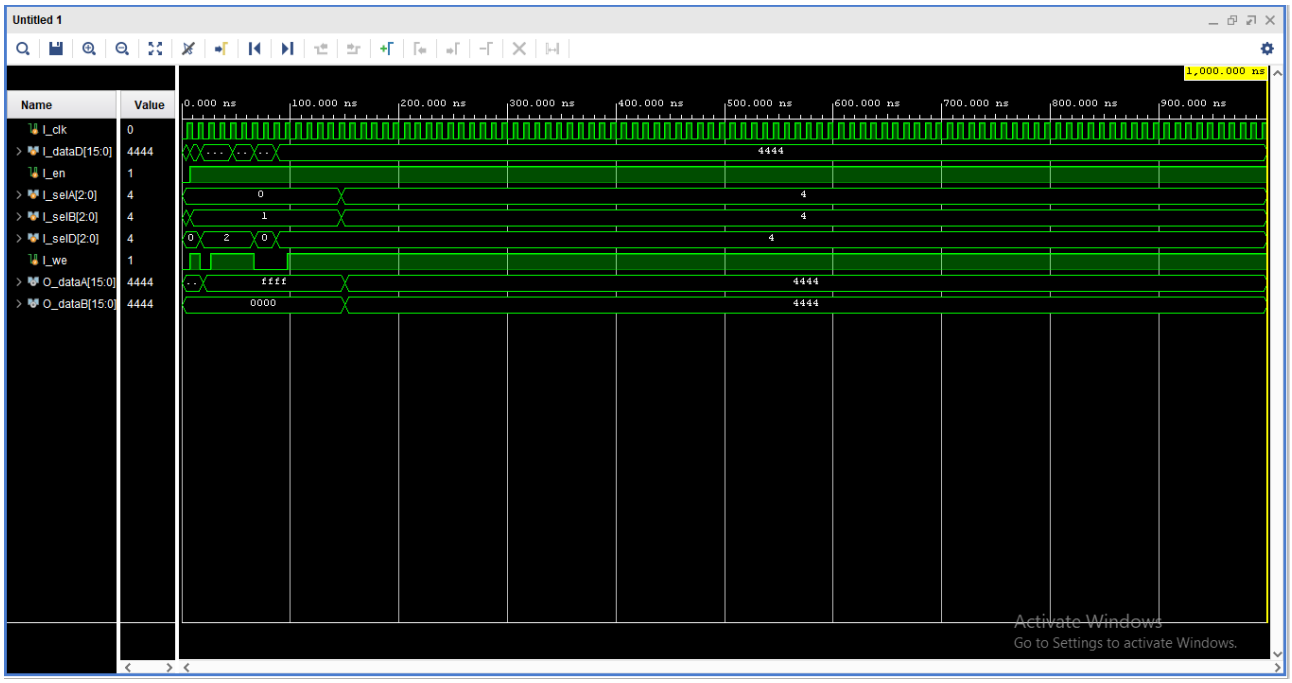
regfile_unittest :

regfile_unittest.v

C:/Users/user/OneDrive/Desktop/OneStop/RISC Processor/RISC Processor.srcs/sim_1/new/regfile_unittest.v

```
`timescale 1ns / 1ps


`timescale 1ns / 1ps

module regfile_unittest();
    reg I_clk;
    reg [15:0] I_dataD;
    reg I_en;
    reg [2:0] I_selA;
    reg [2:0] I_selB;
    reg [2:0] I_selD;
    reg I_we;

    wire [15:0] O_dataA;
    wire [15:0] O_dataB;

    reg_file reg_test(
        I_clk,
        I_en,
        I_we,
        I_selA,
        I_selB,
        I_selD,
        I_dataD,
        O_dataA,
        O_dataB
    );

    initial begin
        I_clk   = 1'b0;
        I_dataD = 0;
        I_en    = 0;
        I_selA  = 0;
        I_selB  = 0;
```

```
        I_selD  = 0;
        I_we    = 0;

        #7;
        I_en    = 1'b1;
        I_selA  = 3'b000;
        I_selB  = 3'b001;
        I_selD  = 3'b000;
        I_dataD = 16'hFFFF;
        I_we    = 1'b1;

        #10;
        I_we    = 1'b0;
        I_selD  = 3'b010;
        I_dataD = 16'h2222;

        #10;
        I_we    = 1;
        #20;
        I_dataD = 16'h3333;

        #20;
        I_we    = 0;
        I_selD  = 3'b000;
        I_dataD = 16'hFEED;

        #20;
        I_selD  = 3'b100;
        I_dataD = 16'h4444;

        #10;
        I_we    = 1;
        #50;
```

```
        I_selA  = 3'b100;
        I_selB  = 3'b100;
    end

    always begin
        #5;
        I_clk = ~I_clk;
    end
endmodule
```

WAVE FORMS :